Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «Основы машинного обучения»
Тема: «Сравнение классических методов классификации»

Выполнил:
Студент 2-го курса
Группы АС-68
Козляковская М.В.
Проверил:
Крошенко А.А.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 3

Wine Quality

• Классифицировать вино на "хорошее" (оценка >= 7) и "обычное" (оценка < 7)

• Задания:

1. Загрузите данные и создайте бинарную целевую переменную;
2. Стандартизируйте признаки и разделите выборку;
3. Обучите модели k-NN, Decision Tree и SVM;
4. Сравните F1-score для каждой модели, так как классы могут быть несбалансированы;
5. Определите, какой алгоритм показал наилучший баланс между точностью и полнотой.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import f1_score

df = pd.read_csv("winequality-white.csv", sep=";")

df.columns = df.columns.str.strip()

df = df.dropna(subset=["quality"])

X = df.drop(columns=["quality"])
y = (df["quality"] >= 7).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify= y
)

knn_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=5))
])

tree_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("classifier", DecisionTreeClassifier(random_state=42))
])
```

```
svm_model = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("classifier", SVC(kernel="rbf"))
])

knn_model.fit(X_train, y_train)
tree_model.fit(X_train, y_train)
svm_model.fit(X_train, y_train)

# Получение предсказаний
knn_pred = knn_model.predict(X_test)
tree_pred = tree_model.predict(X_test)
svm_pred = svm_model.predict(X_test)

# Расчёт F1-score
knn_f1 = f1_score(y_test, knn_pred)
tree_f1 = f1_score(y_test, tree_pred)
svm_f1 = f1_score(y_test, svm_pred)

# Вывод результатов
print("Результаты сравнения моделей:")
print("k-NN F1-score:", round(knn_f1, 3))
print("Decision Tree F1-score:", round(tree_f1, 3))
print("SVM F1-score:", round(svm_f1, 3))

Результаты сравнения моделей:
k-NN F1-score: 0.604
Decision Tree F1-score: 0.636
SVM F1-score: 0.444
```

Model	AUC	CA	F1	Prec	Recall	MCC
kNN	0.847	0.834	0.829	0.826	0.834	0.484
SVM	0.465	0.687	0.706	0.739	0.687	0.223
Tree	0.738	0.844	0.841	0.838	0.844	0.522

Наилучший баланс между точностью (Precision) и полнотой (Recall) показала модель Decision Tree (дерево решений). Её значение F1-score составляет 0,636, что выше, чем у k-NN (0,604) и SVM (0,444). F1-score является гармоническим средним точности и полноты - дерево решений обеспечило наиболее сбалансированное сочетание этих двух метрик среди рассмотренных моделей.